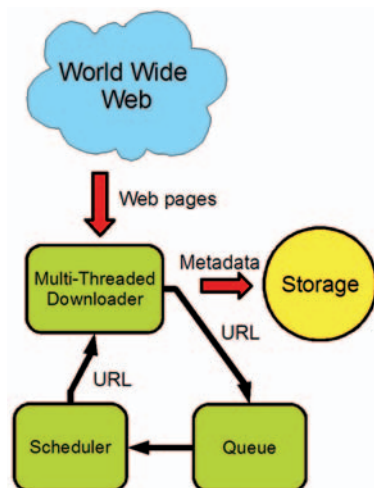




How a spider crawls the web

change rapidly, and so spiders have to keep crawling them over and over again. Static pages are not crawled as frequently. Most sites on the internet strive hard to be dynamic, and offer new content for every visit, which drains a considerable share of the spider's time from new web pages. It takes some time, and a lot of effort on the part of the web designer to get the page to be crawled, which is why many search engines take payments for a site to be crawled, and updated frequently.

Site owners have a love-hate relationship with spiders. While getting their page crawled is important to them for hits, a number of spiders can easily drain resources from a small web site with limited bandwidth. The speed at which spiders can make demands on web sites can take them down and deny the web site's service to legitimate visits. Spiders also use a multi-threaded approach to cache the web pages. This is akin to a download manager making a number of requests to the same site in parallel. Many website owners prefer to disallow some spiders from crawling their content. Therefore, a robot.txt file is used by webmasters to deny some or all spiders access to the page. This is also used so that a web master has the liberty to withhold his data from being crawled. A robot.txt file can be used to allow only some crawlers to search, and disallow other crawlers from searching. A robot.txt file can also be used to protect private content on the web. However, this approach is not always safe, as a number of aggressive crawlers may ignore the robot.txt file and crawl the web page anyway.

The flooded server farms

Search engines require a large amount of storage space. An index of all the web pages crawled by the spider of the search engine is stored. Initially, this index was stored on a single server. Google innovated a technique of storing data in an array of machines, which allowed faster access. This was a cheaper option for Google when it started up in a garage. This technique was soon picked up by other search engines.

The index is constantly updated, and there needs to be several technologies that allow data retrieval from the index. The most common method for this is hashing. Hashing allows data retrieval from incredibly large databases such as those used by search engines. Hashing is the process of creating a relatively small numerical value from a text string. This is used to make retrieval from the index much faster and easier.

For a simplistic explanation of how hashing works, consider a user looking for a particular Linux distribution. In the database of the search engine, there will be a number of distributions such as Fedora, Mandriva, Sabayon, OpenSUSE and so on. If a user searches for "Fedora", the index will have to match the alphanumeric values of all the distributions, of different lengths. A much shorter approach would be the numbers 1, 2, 3, 4 and so on assigned to each of the distributions. When a search is made, the search engine will just match the numerical values of shorter

length, instead of the much longer alphanumeric values. This is just an illustration, and hashing often relies on pretty lengthy (often 32-bit) alphanumeric strings, but the speed of the search is increased in very large databases.

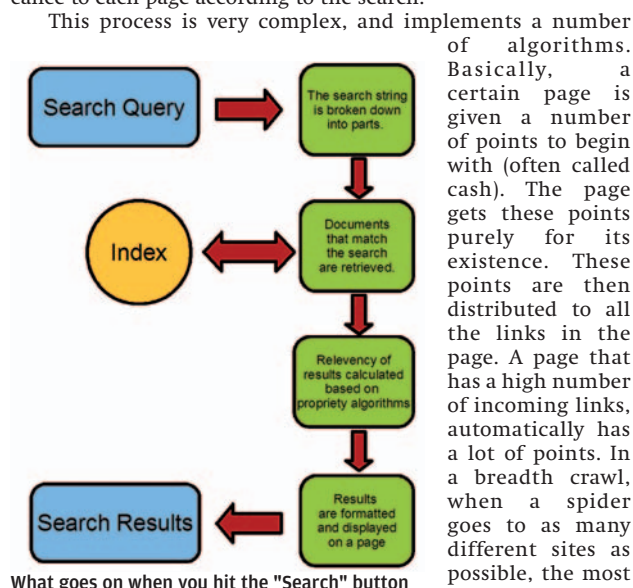
When a spider crawls a page, it reads the page in many ways, and interprets and stores a number of aspects of the page for later use. The spider looks at all the words in the page, their proximity to each other, their presence in bold or in headers, the title of the page and stores the entire page in a cache. If it encounters a non-HTML document such as a PDF, a spreadsheet or a Word file, it stores the same content in XHTML format. A copy of the web page is made on the servers of the crawler, called a *cache*. This cache is the state of the web page when it was crawled, and is useful when the page has been updated, and has lost the content that was relevant to the search. The qualitative information about the page is stored in the servers of the search engine, often after compression. This data is used later to display the search results.

There are three kinds of servers that every major search engine employs. The first kind is the spidering server, that indexes the web. Servers across the globe store parts of the index called "shards". The index functions as an essential whole, but is physically located in different geographical areas. The second kind — document servers, store the cache of the crawled pages. The third kind of servers directs the traffic, resolve search queries, implement search algorithms and interface with the user. Typically, an x86 architecture is used, with a Linux-based OS. A typical search engine may have servers in as many as twenty different locations around the world.

What are you really looking for anyway?

Languages are very complex, and context plays an important role in communication. Words such as "executive", "silver" or "orange" have different meanings and can be interpreted differently. This is a common problem encountered by users, who frequently get search results that are in stark contrast to their expectations. Content creators may have used different words, so a user may have to try different variations of the same search. Those who search for "car" will miss out an entire section of pages that use the word "automobile" instead. Some search engines such as Google resolve this problem by using the tilde character (~) before the search string to search for similar terms.

Apart from the highly interpretive nature of search, there are a number of factors in a page that can affect the relevancy of the content. Therefore, search engines use a number of approaches to resolve the relevancy in each search, and assign a level of significance to each page according to the search.



What goes on when you hit the "Search" button

of algorithms. Basically, a certain page is given a number of points to begin with (often called cash). The page gets these points purely for its existence. These points are then distributed to all the links in the page. A page that has a high number of incoming links, automatically has a lot of points. In a breadth crawl, when a spider goes to as many different sites as possible, the most